Data Encryption Standard (DES): Complete Mathematical Walkthrough

Understanding DES through Complete Step-by-Step Encryption and Decryption

Course

Cryptography and Network Security | B.Tech CS/IT

Test Vector Used Throughout

Plaintext: 0123456789ABCDEF
Key: 133457799BBCDFF1
Ciphertext: 85E813540F0AB405

Learning Objectives

By the end of this session, you will be able to:

1

History & Architecture

Understand why DES was developed, its historical significance, and the 64-bit block / 56-bit key / 16-round Feistel architecture

2

Permutations & Key Gen

Perform Initial Permutation (IP), Final Permutation (FP), and full Key Generation: PC-1, circular shifts, PC-2 → K1–K16

3

F-Function Operations

Apply Expansion (E: 32→48), XOR with round key, S-Box substitution (6→4 bits), and P-Permutation (32→32)

4

Full Cipher + Real-World

Trace a complete Round 1 calculation, understand the Feistel self-inverse property for decryption, and compare DES vs AES

Core Formula

$L_i = R_{i-1}$

Core Formula

$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$

F-Function

$F(R,K) = P(S(E(R) \oplus K))$

History of DES: From Need to Standard

1

1973 — The Problem

U.S. government had no standard encryption algorithm; agencies used incompatible, proprietary ciphers — data sharing was insecure and fragmented

2

1974 — IBM Lucifer

IBM's Horst Feistel designed "Lucifer" — a 128-bit key, 64-bit block cipher; submitted to NBS in response to public call for a standard

3

1975 — NSA Modifications

NSA reduced key size from 128 bits to 56 bits; modified S-Box design (later revealed to strengthen against differential cryptanalysis)

4

1977 — DES Published

Adopted as **FIPS PUB 46**; became the world's first publicly available, government-endorsed encryption standard

5

1998 — EFF Deep Crack

Electronic Frontier Foundation broke DES in **56 hours** for $250,000 — proving the 56-bit key was insufficient for modern security

6

2001 — Replaced by AES

NIST adopted AES with 128/192/256-bit keys; DES officially deprecated; 3DES used as transitional standard until 2024

⚠ DES is **cryptographically broken** — it remains essential study for understanding modern cipher design, but must never be used in new systems.

Why Encryption? The Core Problem

Shannon's Two Properties of Strong Ciphers

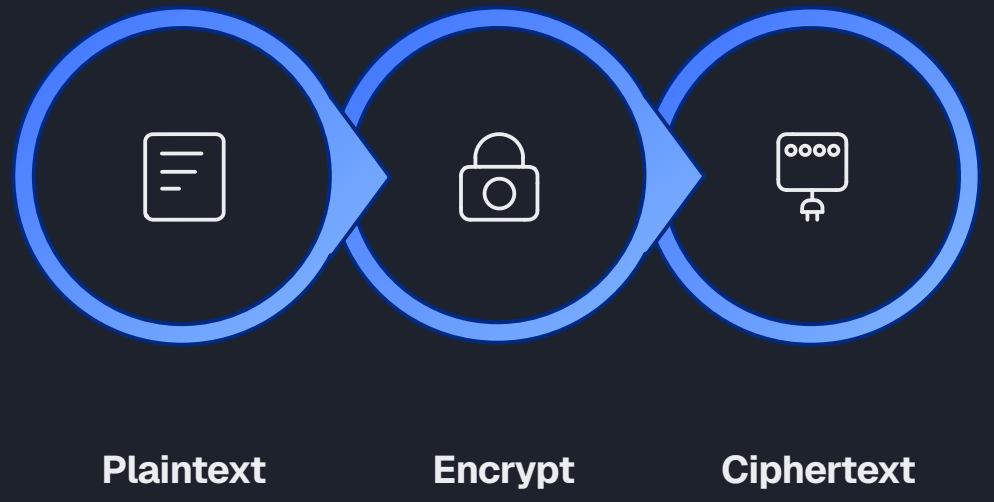
Confusion

Obscure the relationship between the key and ciphertext — implemented via S-Boxes

Diffusion

Spread plaintext statistics across the ciphertext — implemented via permutations

The Symmetric Encryption Model



DES uses the **same 56-bit key** for both encryption and decryption — the key must be shared securely between sender and receiver beforehand.

The Threat

Data transmitted over networks can be intercepted — without encryption, plaintext is readable by anyone who captures the packet

The Goal

Without the key, an attacker seeing only ciphertext should be computationally unable to recover plaintext — even with unlimited time

DES Achievement

Achieves both confusion (non-linear S-Boxes) and diffusion (IP, FP, E, P permutations) — the foundation of all modern block ciphers

DES Overview: The Big Picture

01

Input: 64-bit Plaintext

Example hex: `0123456789ABCDEF` — DES processes data in 64-bit (8-byte) chunks

02

Initial Permutation (IP)

Rearranges all 64 bits according to a fixed IP table — no cryptographic strength; aids hardware implementation

03

Split → L0 and R0

IP output split into Left half L0 (bits 1–32) and Right half R0 (bits 33–64) — each 32 bits

04

16 Feistel Rounds

Each round: `Li = Ri-1`, `Ri = Li-1 ⊕ F(Ri-1, Ki)` — uses a unique 48-bit round key Ki

05

Final Swap + FP (IP^{-1})

After Round 16: combine as `R16 // L16` → apply FP → 64-bit ciphertext: `85E813540F0AB405`

Key Schedule (parallel)

Original 64-bit key → **PC-1** → C0, D0 → **16× left circular shifts** → **PC-2** → K1 through K16 (each 48 bits)

Key Numbers to Remember

Block: **64 bits** | Key input: **64 bits** | Effective key: **56 bits** | Rounds: **16** | Round key: **48 bits** | Keyspace: 2^{56}

DES Block Diagram: Complete Architecture

Data Path

1. **Plaintext (64 bits)**
   Input block enters algorithm
2. **IP (Initial Permutation)**
   Fixed bit rearrangement → L0 ∥ R0
3. **16 Feistel Rounds**
   $L_i = R_{i-1}$; $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$ for i=1…16
4. **Final Swap + FP**
   $R_{16} \| L_{16} \rightarrow IP^{-1} \rightarrow$ Ciphertext (64 bits)

Key Schedule (Right-side feed)

1. **64-bit Key Input**
   Hex: `133457799BBCDFF1`
2. **PC-1 → C0, D0**
   56 bits selected; split into two 28-bit halves
3. **16× Shifts + PC-2**
   Each iteration: LSi shift → PC-2 selects 48 bits → Ki
4. **K1 … K16**
   Each 48-bit Ki feeds into its corresponding round

The key schedule runs **in parallel** with data processing — by the time Round i begins, Ki is already computed and ready.

DES Parameters: Key Numbers to Know

64

Block Size (bits)

DES encrypts in 64-bit chunks; longer messages use modes: ECB, CBC, CFB, OFB

56

Effective Key (bits)

Every 8th bit (positions 8,16,24,32,40,48,56,64) is a parity bit — discarded before use

16

Feistel Rounds

Each round uses a different 48-bit subkey derived from the master key

48

Round Key (bits)

Each of the 16 round keys is 48 bits, derived from the 56-bit key via PC-2

2^{56}

Keyspace

≈72 quadrillion possible keys — sufficient in 1977; broken by brute force in 1998

32

F-Function Output (bits)

F takes 32-bit R and 48-bit K; outputs 32 bits via E → XOR → S-Boxes → P

Parity Bits (8 bits discarded)

Positions 8, 16, 24, 32, 40, 48, 56, 64 — used for error detection in key transmission (each byte has odd parity). Not used in encryption.

Why 56 bits instead of 64?

NSA's controversial 1975 decision — later justified as intentional hardware optimization. The 8 parity bits allowed key transmission error detection. Differential cryptanalysis (discovered publicly 1990) showed the S-Boxes were actually strengthened.

Feistel Structure: The Heart of DES

Round Operation

$L_i = R_{i-1}$

New Left = Old Right (simple pass-through)

$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$

New Right = Old Left XOR'd with F-function result

F-Function Internals

01

Expansion E

32 → 48 bits (duplicate boundary bits)

02

XOR with Ki

48-bit XOR with round key

03

S-Boxes (×8)

48 → 32 bits via non-linear substitution

04

P-Permutation

32 → 32 bits (diffusion)

Why Feistel? The Key Insight

Only **half** the data (R) is processed by F each round — the other half (L) is simply passed through. This means F does NOT need to be invertible.

Invertibility Proof

Given Li and Ri from encryption, recover:
$L_{i-1} = R_i$
$R_{i-1} = L_i \oplus F(R_i, K_i)$
No need to compute F^{-1} — XOR handles it!

Hardware Advantage

Same circuit performs both encryption and decryption — simply reverse the key loading order. Reduces chip cost by ~50%.

Visual Metaphor

Each round looks like a **butterfly** — R crosses over to become new L; L XOR'd with F(R,K) becomes new R

Encryption Formulas: The Mathematical Core

Pre-Round: Initial Permutation

$IP(Plaintext) \rightarrow L0 \mathbin{//} R0$
Splits 64 bits into two 32-bit halves

Feistel Round Equations (i = 1 to 16)

$Li = Ri\text{-}1$ — Left output = Right input
$Ri = Li\text{-}1 \oplus F(Ri\text{-}1, Ki)$ — Right output = Left input XOR F-result

F-Function Expansion (full form)

$F(Ri\text{-}1, Ki) = P(S(E(Ri\text{-}1) \oplus Ki))$
E: 32→48 bits | ⊕Ki: XOR | S(·): 48→32 bits | P(·): 32→32 bits

Post-Round: Final Permutation

After Round 16: combine as $R16 \mathbin{//} L16$ (right first!)
$Ciphertext = FP(R16 \mathbin{//} L16) = IP^{-1}(R16 \mathbin{//} L16)$

Key Schedule Formula

$Ki = PC\text{-}2(LSi(Ci\text{-}1) \mathbin{//} LSi(Di\text{-}1))$
LSi = left circular shift by schedule amount (1 or 2 positions)

Decryption (Self-Inverse Property)

Same algorithm with keys in reverse: K16, K15, ..., K1
$DEC_K(ENC_K(P)) = P$ for all P, K — guaranteed by Feistel

Initial Permutation (IP): Why and What

Purpose & Design Rationale

IP rearranges all 64 input bits into a new order before the 16 rounds begin. It provides **no cryptographic security** on its own — it was designed to facilitate byte-oriented hardware loading in 1970s DES chip implementations.

How to Read the IP Table

Position *i* in the output takes the bit from position `IP[i]` of the input. Example: Output bit 1 = input bit **58**; Output bit 2 = input bit **50**

Pattern

IP interleaves **even-numbered bits** (rows 1–4) and **odd-numbered bits** (rows 5–8) — separating them for hardware efficiency.

Inverse Relationship

FP = IP^{-1} undoes this permutation at the end: `FP[IP[i]] = i` for all i = 1…64

> Security comes entirely from the 16 rounds and the secret key — not from IP or FP.

IP Table (8×8 Grid)

Out 1–8	Out 9–16	Out 17–24	Out 25–32	Out 33–40	Out 41–48	Out 49–56	Out 57–64
58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

IP Example: Hex to Binary Conversion

WORKED EXAMPLE — PLAINTEXT: 0123456789ABCDEF

Step 1 — Convert Each Hex Digit to 4-bit Binary

Hex	Binary	Hex	Binary	Hex	Binary	Hex	Binary
0	0000	4	0100	8	1000	C	1100
1	0001	5	0101	9	1001	D	1101
2	0010	6	0110	A	1010	E	1110
3	0011	7	0111	B	1011	F	1111

Step 2 — Full 64-bit Binary Plaintext

```
0000 0001  0010 0011  0100 0101  0110 0111  1000 1001  1010 1011  1100 1101  1110 1111
 0    1     2    3     4    5     6    7     8    9     A    B     C    D     E    F
```

Bit Numbering Reference

Bit 1 (leftmost) = 0 | Bit 8 = 1 | Bit 9 = 0 | Bit 16 = 0 | Bit 32 = 1 | Bit 58 = 1 | Bit 64 (rightmost) = 1

Next step: Apply IP table — output bit 1 = input bit 58 = 1; output bit 2 = input bit 50 = 1; continue for all 64 positions

IP Example: Applying the Permutation

ROW-BY-ROW IP APPLICATION

Input (64-bit binary, bits numbered 1–64)

```
00000001 00100011 01000101 01100111 10001001 10101011 11001101 11101111
```

Applying IP — First 16 Output Bits

Output Position	Reads Input Bit	Bit Value	Output Group
Out 1	Bit 58	`1`	Row 1 → `11110000`
Out 2	Bit 50	`1`	
Out 3	Bit 42	`1`	
Out 4	Bit 34	`1`	
Out 5	Bit 26	`0`	
Out 6	Bit 18	`0`	
Out 7	Bit 10	`0`	
Out 8	Bit 2	`0`	Row 2 → `11110000`

Full IP Output (L0 ∥ R0)

L0: 11001100 00000000 11001100 11111111 (bits 1–32)
R0: 11110000 10101010 11110000 10101010 (bits 33–64)

L0 (hex)

`CC00CCFF`

R0 (hex)

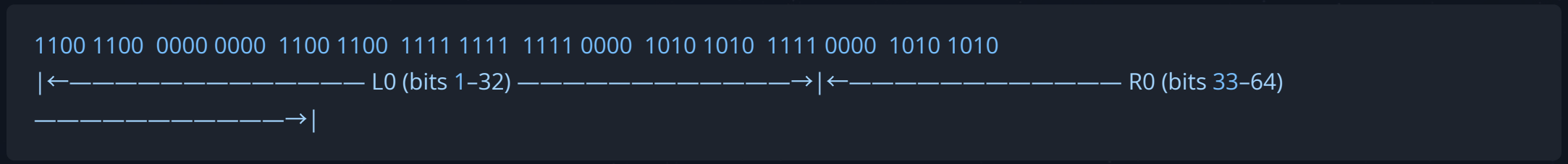
`F0AAF0AA`

Next

L0 and R0 enter **Round 1** of the Feistel structure

IP Example: Verification and Split

Verified IP Output — Complete 64 Bits



L0 = bits 1–32

1100 1100 0000 0000 1100 1100 1111 1111

Hex: CC00CCFF

R0 = bits 33–64

1111 0000 1010 1010 1111 0000 1010 1010

Hex: F0AAF0AA

Key Insight: What IP Does

IP has separated the **even-position bits** from **odd-position bits** — this interleaving pattern is intentional for hardware byte-loading efficiency in 1970s chips.

Security Reminder

IP provides **zero cryptographic security** — an attacker who knows DES knows the IP table. Security comes entirely from the 16 rounds and the secret key.

Ready for Round 1

Values L0 = CC00CCFF and R0 = F0AAF0AA are the starting state for all 16 Feistel rounds.

Key Generation: Starting with the Original Key

KEY: 133457799BBCDFF1

Step 1 — Convert Key to 64-bit Binary

Hex	Bin	Hex	Bin	Hex	Bin	Hex	Bin
1	0001	5	0101	9	1001	D	1101
3	0011	7	0111	B	1011	F	1111
3	0011	7	0111	B	1011	F	1111
4	0100	9	1001	C	1100	1	0001

Full 64-bit Key Binary

00010011 00110100 01010111 01111001 10011011 10111100 11011111 11110001

Parity Bits (discarded)

Positions 8, 16, 24, 32, 40, 48, 56, 64 are parity bits: `1, 0, 1, 1, 1, 0, 1, 1`
These are **discarded** before key processing begins — not used in any round key.

Next Step: PC-1

Apply **PC-1 (Permuted Choice 1)** to select the 56 non-parity bits and rearrange them into two 28-bit halves **C0** and **D0**.

Key Generation: PC-1 Table and 56-bit Key

PC-1 Purpose

Selects 56 bits from the 64-bit key (discarding all 8 parity bits at positions 8, 16, 24, 32, 40, 48, 56, 64) and permutes them into two 28-bit halves C0 and D0.

PC-1 Table

C0 R1	C0 R2	C0 R3	C0 R4	D0 R1	D0 R2	D0 R3	D0 R4
57	49	41	33	25	17	9	1
58	50	42	34	26	18	10	2
59	51	43	35	27	19	11	3
60	52	44	36	63	55	47	39
31	23	15	7	62	54	46	38
30	22	14	6	61	53	45	37
29	21	13	5	28	20	12	4

PC-1 Applied to Example Key

C0 (28 bits):

```
1111000 0110011 0010101 0101111
```

D0 (28 bits):

```
0101010 1011001 1001111 0001111
```

C0 (hex approx): `F0CAF`

D0 (hex approx): `5AC7F`

These C0 and D0 halves are the **starting point** for generating all 16 round keys K1 through K16.

Key Generation: Left Circular Shift Schedule

Shift Schedule — All 16 Rounds

Round	Shift	Cumulative	Round	Shift	Cumulative
1	**1**	1	9	**1**	15
2	**1**	2	10	2	17
3	2	4	11	2	19
4	2	6	12	2	21
5	2	8	13	2	23
6	2	10	14	2	25
7	2	12	15	2	27
8	2	14	16	**1**	28

Concept

Each round, both C and D halves are independently **left-rotated** (circular shift) by 1 or 2 positions according to the fixed schedule above.

Total Shifts

4×1 + 12×2 = 4 + 24 = **28 positions** — exactly one full rotation of the 28-bit register. Every key bit is used!

Round 1 Example (shift by 1)

C0: `11110000110011001010101011111`
C1: `11100001100110010101010111111`

D0: `01010101011001100111100011111`
D1: `10101010110011001111000111110`

After each shift: Ci // Di (56 bits) → fed into **PC-2** → round key Ki

Left Circular Shift: Rounds 1–8 Worked Example

C-HALF PROGRESSION (28-BIT REGISTER)

State	Shift	28-bit C Value
C0	—	1111000 0110011 0010101 0101111
C1	1	1110000 1100110 0101010 1011111
C2	1	1100001 1001100 1010101 0111111
C3	2	0000110 0110010 1010101 1111111
C4	2	0011001 1001010 1010111 1111100
C5	2	1100110 0101010 1011111 1110000
C6	2	0011001 0101010 1111111 1000011
C7	2	1100101 0101011 1111110 0001100
C8	2	0010101 0101111 1111000 0110011

Pattern: Each shift rotates the leftmost bit(s) to the rightmost position(s) — the 28-bit register wraps around. D halves shift identically following the same schedule independently.

Left Circular Shift: Rounds 9–16

C-HALF PROGRESSION — COMPLETING THE FULL 28-POSITION ROTATION

State	Shift	28-bit C Value
C8	—	0010101 0101111 1111000 0110011
C9	1	0101010 1011111 1110000 1100110
C10	2	0101010 1111111 1000011 0011001
C11	2	0101011 1111110 0001100 1100101
C12	2	0101111 1111000 0110011 0010101
C13	2	0111111 1100000 1100110 0101010
C14	2	1111111 0000011 0011001 0101010
C15	2	1111100 0001100 1100101 0101011
C16	**1**	

PC-2: Generating Round Keys K1–K16

PC-2 Table (48 positions from 56-bit Ci // Di)

Col 1	Col 2	Col 3	Col 4	Col 5	Col 6	Col 7	Col 8
14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

PC-2 Purpose

Selects **48 bits** from the 56-bit combined Ci // Di to form round key Ki — discards 8 bits per round (different bits discarded each round due to shifting). Positions 1–28 come from Ci, positions 29–56 from Di.

Applying PC-2 to C1 // D1 → K1

K1 (48 bits, grouped as 8×6):

```
000110 110000 001011
101111 111111 000111
000001 110010
```

K1 hex: `1B02EFFC7072`

Repeat for Rounds 2–16: each Ci // Di pair produces a unique Ki via the same PC-2 table

PC-2: Round Keys K1–K8 Summary

ALL VALUES ARE 48-BIT BINARY, GROUPED AS 8 × 6 BITS

Key	48-bit Value (8 groups of 6)	Hex
K1	000110 110000 001011 101111 111111 000111 000001 110010	1B02EFFC7072
K2	011110 011010 111011 011001 110110 111100 100111 100101	79AED9DBC9E5
K3	010101 011111 110010 001010 010000 101100 111110 011001	55FC8A50BCE9
K4	011100 101010 110111 010110 110110 110011 010100 011101	72AED6D334D5
K5	011111 001110 110000 000111 111010 110101 001110 101000	7CE807EA72E8
K6	011000 111010 010100 111110 010100 000111 101100 101111	63A94E07ACB7
K7	111011 001000 010010 110111 111101 100001 100010 111100	EC8976FD88BC
K8	111101 111000 101000 111010 110000 010011 101111 111011	F7A2C04FBFEB

Each key is 48 bits = 8 groups of 6 bits — the 6-bit grouping aligns exactly with the 8 S-Box inputs in the F-function.

PC-2: Round Keys K9–K16 Summary

COMPLETING THE FULL KEY SCHEDULE

Key	48-bit Value (8 groups of 6)
K9	111100 001101 000111 100001 111001 001101 100111 000110
K10	111001 000110 001101 001111 010011 111100 001010 011101
K11	101111 111001 000110 001101 001111 010011 111100 001010
K12	001101 001111 010011 111100 001010 011101 101111 111001
K13	001111 010011 111100 001010 011101 101111 111001 000110
K14	010011 111100 001010 011101 101111 111001 000110 001101
K15	111100 001010 011101 101111 111001 000110 001101 001111
K16	110010 110011 110110 001011 000011 100001 011111 110101

Key Schedule Complete

All 16 round keys K1–K16 are now ready — key generation is finished

Decryption Order

Apply keys in reverse: **K16, K15, K14, ..., K2, K1** — same algorithm, reversed schedule

Key Uniqueness

No two round keys are identical (for non-weak keys) — each Ci shift produces distinct bits

Expansion Function (E): 32 → 48 Bits

E-Table (8 rows × 6 columns)

Bit 1	Bit 2	Bit 3	Bit 4	Bit 5	Bit 6
32	1	2	3	4	**5**
4	5	6	7	8	**9**
8	9	10	11	12	**13**
12	13	14	15	16	**17**
16	17	18	19	20	**21**
20	21	22	23	24	**25**
24	25	26	27	28	**29**
28	29	30	31	32	**1**

Bold = duplicated boundary bits; each row = one 6-bit S-Box input

Purpose

Expands the 32-bit right half (Ri-1) to 48 bits so it can be XOR'd with the 48-bit round key Ki.

Method

32 bits divided into 8 groups of 4 bits. Each group expands to 6 bits by **borrowing 1 bit from the adjacent group** on each side — creating overlap.

Duplicated Bits

Bits 1, 4, 5, 8, 9, 12, 13, 16, 17, 20, 21, 24, 25, 28, 29, 32 each appear **twice**. This is what causes 32 → 48 expansion.

Avalanche Effect

Changing 1 bit in Ri-1 affects **2 adjacent S-Box inputs**, spreading changes across the output — critical for diffusion.

Expansion Function: Worked Example

INPUT: R0 = F0AAF0AA

Input R0 (32 bits)

```
1111 0000  1010 1010  1111 0000  1010 1010
Bit:  1234   5678   9..12  13..16  17..20  21..24  25..28  29..32
```

E-Table Application — Row by Row

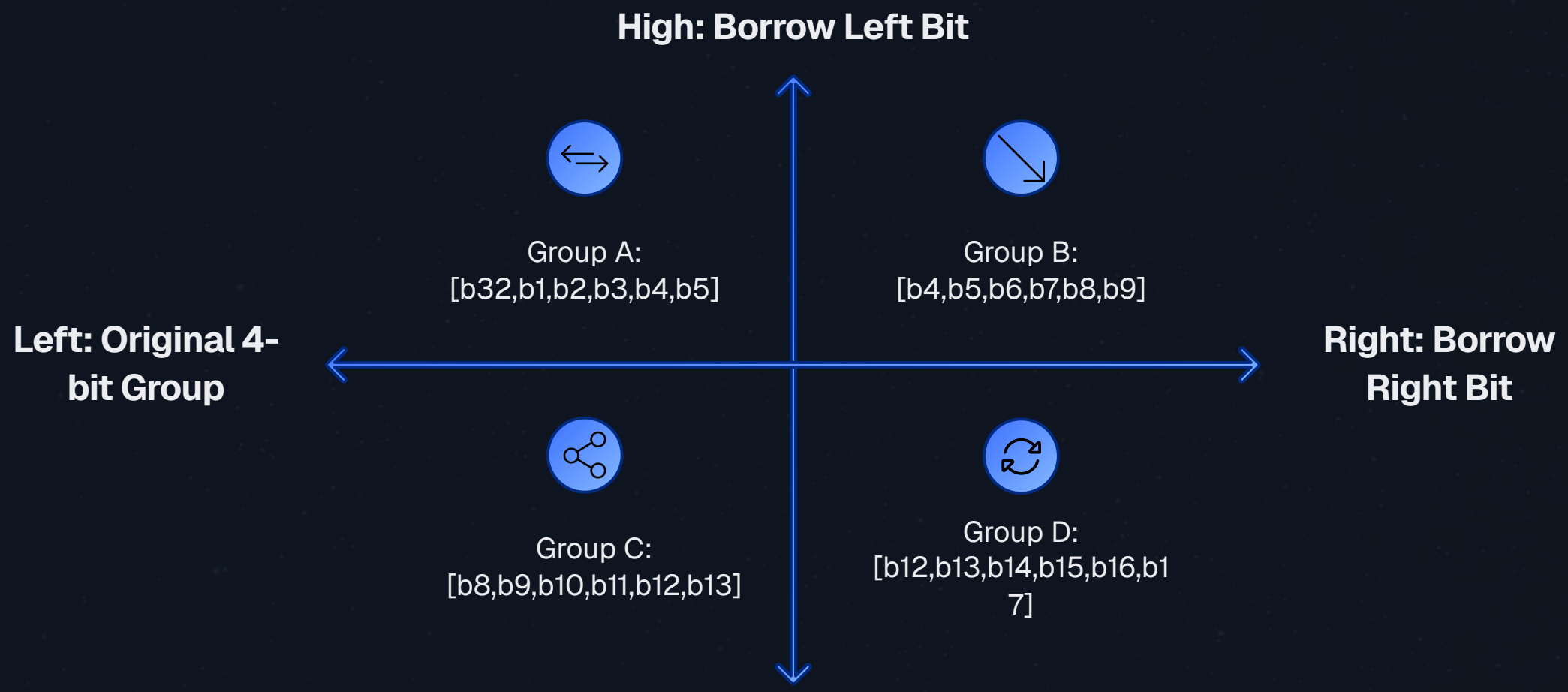
Row	Input Positions	Bits Selected	6-bit Output
1	32,1,2,3,4,5	0,1,1,1,1,0	011110
2	4,5,6,7,8,9	1,0,0,0,0,1	100001
3	8,9,10,11,12,13	0,1,0,1,0,1	010101
4	12,13,14,15,16,17	0,1,0,1,0,1	010101
5	16,17,18,19,20,21	0,1,1,1,1,0	011110
6	20,21,22,23,24,25	1,0,0,0,0,1	100001
7	24,25,26,27,28,29	0,1,0,1,0,1	010101
8	28,29,30,31,32,1	0,1,0,1,0,1	010101

E(R0) = 48 bits

```
011110 100001 010101 010101 011110 100001 010101 010101
```

Expansion Function: Visualizing Bit Duplication

Structure of Each 4-bit Group → 6-bit Expansion



Group 1 Expansion Detail

Input: [b1, b2, b3, b4] from the 32-bit block
Output: [**b32**, b1, b2, b3, b4, **b5**]
Borrows b32 from end (wrap) and b5 from next group

Group 2 Expansion Detail

Input: [b5, b6, b7, b8]
Output: [**b4**, b5, b6, b7, b8, **b9**]
Borrows b4 from previous and b9 from next

Avalanche Chain Reaction

If bit 4 changes →

- Affects Group 1 output (position 5) → changes S-Box 1 input
- Affects Group 2 output (position 1) → changes S-Box 2 input
- Two different S-Boxes receive different inputs → multiple output bits change

---

Final E(R0) Ready for XOR

`011110 100001 010101 010101 011110 100001 010101 010101`

48 bits → now XOR'd with 48-bit round key K1

XOR Operation: Truth Table and Rules

XOR Truth Table (Symbol: ⊕)

A	B	A ⊕ B	Rule
0	0	**0**	Same bits → 0
0	1	**1**	Different bits → 1
1	0	**1**	Different bits → 1
1	1	**0**	Same bits → 0

Self-Inverse Property (Critical!)

If `A ⊕ B = C`, then `C ⊕ B = A`
This is **exactly** why DES decryption works with the same algorithm — XOR undoes itself.

Two Uses of XOR in DES

Inside F-function

`E(Ri-1) ⊕ Ki`
48-bit XOR with round key

Feistel Combination

`Li-1 ⊕ F(Ri-1, Ki)`
Produces new Ri

Why XOR?

- **Fast:** Single gate in hardware, one instruction in software
- **Reversible:** Perfectly invertible without knowing the other operand's inverse
- **Confusion:** Changing 1 key bit flips exactly 1 bit in the XOR output
- **Bit-by-bit:** No carry, no interaction between positions

XOR Operation: E(R0) ⊕ K1 Worked Example

48-BIT XOR — ALL 8 GROUPS

Inputs

E(R0): 011110 100001 010101 010101 011110 100001 010101 010101
K1: 000110 110000 001011 101111 111111 000111 000001 110010

Group	E(R0) bits	K1 bits	XOR Result	→ S-Box
1	011110	000110		

XOR Operation: Verification and Next Step

Full XOR Result Summary (48 bits, 8 groups of 6)

Group 1 → S1	Group 2 → S2	Group 3 → S3	Group 4 → S4
011000	010001	011110	111010

Group 5 → S5	Group 6 → S6	Group 7 → S7	Group 8 → S8
100001	100110	010100	100111

Next Step

Each 6-bit group is fed into its corresponding S-Box to produce a **4-bit output** — total 8 × 4 = **32 bits**. This is where DES's non-linearity lives.

Verification: XOR Self-Inverse

$(E(R0) \oplus K1) \oplus K1 = E(R0)$ ✓

This property guarantees that the decryption process can recover E(R0) simply by XOR-ing with K1 again — the same operation, no inverse needed.

S-Boxes: The Only Non-Linear Component

Why S-Boxes?

All other DES operations (permutations, XOR, shifts) are **linear** — S-Boxes provide the **non-linearity** that makes DES resistant to linear cryptanalysis. Without S-Boxes, DES would be breakable with simple algebra.

Structure

8 S-Boxes (S1–S8), each takes **6-bit input → 4-bit output**

Non-Linearity Property

Changing 1 input bit can change 1–4 output bits **unpredictably** — this is the core of DES security. No linear equation can describe S-Box behavior.

NSA Design

S-Box values were modified by NSA in 1975 — in 1990, Biham & Shamir showed the modifications actually *strengthened* DES against differential cryptanalysis unknown to the public at the time.

How to Use an S-Box: 3-Step Lookup

01

Get 6-bit Input

Input: b1 b2 b3 b4 b5 b6 (from E(R) ⊕ K)

02

Find Row

Row = b1 b6 (first and last bits) → 2-bit binary → row 0, 1, 2, or 3

03

Find Column

Column = b2 b3 b4 b5 (middle 4 bits) → 4-bit binary → column 0–15

Example: S-Box 1, input = 011000

b1=**0**, b6=**0** → Row = 00 = **Row 0**
b2b3b4b5 = 1100 = 12 → **Column 12**
S1[0][12] = **5** → binary = `0101`

S-Box 1: Complete Table

S1 — ROW B1B6 (0–3) × COLUMN B2B3B4B5 (0–15) → 4-BIT OUTPUT

Row\ Col	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13	1	2	15	11	8	3	10	6	12	**5**	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

Lookup Example 1 (our Round 1)

Input: `011000`
Row = b1b6 = **00** = Row 0
Col = b2b3b4b5 = **1100** = Col 12
S1[0][12] = **5** → `0101`

Lookup Example 2 (S2 demo)

Input to S2: 010001
Row = b1b6 = **01** = Row 1
Col = b2b3b4b5 = **1000** = Col 8
S2[1][8] = **12** → 1100

S-Box Lookup: Row/Column Mechanics Deep Dive

Why b1 and b6 for the Row?

The outer bits (b1 and b6) of each 6-bit group are the **borrowed boundary bits** from the expansion function — they are the bits most likely to cause collisions between neighboring groups. Using them as row selectors ensures maximum mixing between adjacent S-Box computations.

1

Extract 6-bit Group

From E(R) ⊕ K: one 6-bit chunk per S-Box

2

Isolate b1 and b6

Form 2-bit row index: `row = 2×b1 + b6`

3

Isolate b2–b5

Form 4-bit column index: `col = 8×b2 + 4×b3 + 2×b4 + b5`

4

Table Lookup

Read S-Box value at [row][col] — output is 4-bit binary (values 0–15)

Complete S-Box Lookup Reference (Round 1)

S-Box	Input	b1b6 (Row)	b2b3b4b5 (Col)	Output Value	4-bit Binary
S1	`011000`	00 = Row 0	1100 = Col 12	5	`0101`
S2	`010001`	01 = Row 1	1000 = Col 8	12	`1100`

S-Boxes 2–4: Complete Tables

S-Box 2

R\C	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
1	3	13	4	7	15	2	8	14	**12**	0	1	10	6	9	11	5
2	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
3	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9

S-Box 3

R\C	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	**8**
1	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
2	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
3	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12

S-Box 4

R\C	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
1	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
2	10	6	9	0	12	11	7	13	15	1	3	14	5	**2**	8	4
3	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14

S-Boxes 5–8: Complete Tables

S-Box 5

R\C	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
1	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
2	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
3	**11**	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3

S-Box 6

R\C	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
1	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
2	9	14	15	**5**	2	8	12	3	7	0	4	10	1	13	11	6
3	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13

S-Box 7

R\C	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	4	11	2	14	15	0	8	13	3	12	**9**	7	5	10	6	1
1	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
2	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
3	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12

S-Box 8

R\C	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
1	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
2	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
3	2	1	14	**7**	4	10	8	13	15	12	9	0	3	5	6	11

S-Box Lookups: All 8 Groups for Round 1

XOR INPUT: 011000 | 010001 | 011110 | 111010 | 100001 | 100110 | 010100 | 100111

S-Box	6-bit Input	b1b6 → Row	b2b3b4b5 → Col	Table Value	4-bit Output
S1	011000	00 → Row 0	1100 → Col 12	**5**	0101
S2	010001	01 → Row 1	1000 → Col 8	**12**	1100
S3	011110	00 → Row 0	1111 → Col 15	**8**	1000
S4	111010	10 → Row 2	1101 → Col 13	**2**	0010
S5	100001	11 → Row 3	0000 → Col 0	**11**	1011
S6	100110	10 → Row 2	0011 → Col 3	**5**	0101
S7	010100	00 → Row 0	1010 → Col 10	**9**	1001
S8	100111	11 → Row 3	0011 → Col 3	**7**	0111

S-Box Output (32 bits — concatenation of all 8 four-bit outputs)

```
0101 1100 1000 0010 1011 0101 1001 0111
 S1   S2   S3   S4   S5   S6   S7   S8
```

This 32-bit result is passed to the **P-Permutation** — the final step of the F-function.

P-Permutation: Diffusing S-Box Output

P-Table (4 rows × 8 columns)

Col 1	Col 2	Col 3	Col 4	Col 5	Col 6	Col 7	Col 8
16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

Purpose

Permutes the 32-bit S-Box output to **spread (diffuse)** each S-Box's output bits across multiple S-Box inputs in the *next* round — ensures the avalanche effect propagates.

Worked Calculation

S-Box output (input to P):

```
0101 1100 1000 0010
1011 0101 1001 0111
```

Apply P-table — select bits by position:

- Out 1 = input bit 16 = 0
- Out 2 = input bit 7 = 1
- Out 3 = input bit 20 = 0
- Out 4 = input bit 21 = 1 … (continue all 32)

F(R0, K1) Output (32 bits)

```
0010 0011 0100 1010
1010 1001 1011 1011
```

This is the **complete output of F(R0, K1)** — ready to be XOR'd with L0 to produce R1

P-Permutation: Verification and Diffusion Analysis

F(R0, K1) Confirmed

F(R0, K1) = 0010 0011 0100 1010 1010 1001 1011 1011

Computing R1 = L0 ⊕ F(R0, K1)

Value	Bits 1–16	Bits 17–32
L0	1100 1100 0000 0000	1100 1100 1111 1111
F(R0,K1)	0010 0011 0100 1010	1010 1001 1011 1011
R1 = L0⊕F		

Complete Round 1: Full Walkthrough Summary

NIST TEST VECTOR — VERIFIED AGAINST PUBLISHED DES STANDARD

Step	Operation	Result (48 or 32 bits)
Inputs	L0, R0, K1	L0=CC00CCFF, R0=F0AAF0AA, K1=1B02EFFC7072
Step 1	Expansion E(R0)	011110 100001 010101 010101 011110 100001 010101 010101
Step 2	E(R0) ⊕ K1	011000 010001 011110 111010 100001 100110 010100 100111
Step 3	S-Box substitution (8 lookups)	0101 1100 1000 0010 1011 0101 1001 0111 (32 bits)
Step 4	P-Permutation	0010 0011 0100 1010 1010 1001 1011 1011 = F(R0,K1)
Step 5	R1 = L0 ⊕ F(R0,K1)	1110 1111 0100 1010 0110 0101 0100 0100 = EF4A6544
Step 6	L1 = R0 (pass-through)	1111 0000 1010 1010 1111 0000 1010 1010 = F0AAF0AA

Round 1 Output

L1 = F0AAF0AA
R1 = EF4A6544

Enter Round 2

L1, R1 feed into Round 2 with key **K2** = 79AED9DBC9E5

Pattern Repeats

Same 6-step sequence for each of the 16 rounds — only the round key Ki changes

Rounds 2–8: Progressive Feistel Iterations

Round Equations (all follow the same pattern)

Round	Li (left output)	Ri (right output)	Key Used
2	L2 = R1 = EF4A6544	R2 = L1 ⊕ F(R1, K2)	K2
3	L3 = R2	R3 = L2 ⊕ F(R2, K3)	K3
4	L4 = R3	R4 = L3 ⊕ F(R3, K4)	K4
5	L5 = R4	R5 = L4 ⊕ F(R4, K5)	K5
6	L6 = R5	R6 = L5 ⊕ F(R5, K6)	K6
7	L7 = R6	R7 = L6 ⊕ F(R6, K7)	K7
8	L8 = R7	R8 = L7 ⊕ F(R7, K8)	K8

After 8 Rounds

The data is thoroughly mixed — each output bit depends on **all 64 input bits** and **all 56 key bits**. The avalanche effect is fully established by Round 5.

Key Insight: Feistel Guarantee

Even if F is not invertible on its own, the entire round IS invertible. Given Li and Ri, you can always recover Li-1 = Ri and Ri-1 = Li ⊕ F(Ri, Ki). This is critical for decryption.

Rounds 9–16: Completing the Cipher

Round	Li	Ri	Key
9	L9 = R8	R9 = L8 ⊕ F(R8, K9)	K9
10	L10 = R9	R10 = L9 ⊕ F(R9, K10)	K10
11	L11 = R10	R11 = L10 ⊕ F(R10, K11)	K11
12	L12 = R11	R12 = L11 ⊕ F(R11, K12)	K12
13	L13 = R12	R13 = L12 ⊕ F(R12, K13)	K13
14	L14 = R13	R14 = L13 ⊕ F(R13, K14)	K14
15	L15 = R14	R15 = L14 ⊕ F(R14, K15)	K15
16	L16 = R15	R16 = L15 ⊕ F(R15, K16)	K16

After Round 16 — Known Values (NIST Test Vector)

L16

`0100 0011 0100 0010 0011 0010 0011 0100`
Hex: `43423234`

R16

`0000 1010 0100 1100 1101 1001 1001 0101`
Hex: `0A4CD995`

Next: Final Swap

Combine as `R16 // L16` before applying FP (IP^{-1})

Final Swap: Why It's Necessary

The Problem Without Swap

After Round 16, the last operation was:
$R16 = L15 \oplus F(R15, K16)$
The left half L16 = R15 was **never processed by F** in Round 16 — it was just passed through. Without a swap, encryption and decryption would require different final steps.

The Solution

Before applying FP, combine the halves as

Final Permutation (FP = IP^{-1}): Complete Table

FP Table (8×8 Grid)

Out 1–8	Out 9–16	Out 17–24	Out 25–32	Out 33–40	Out 41–48	Out 49–56	Out 57–64
40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

FP is the Exact Inverse of IP

If IP moved bit *i* to position *j*, then FP moves bit *j* back to position *i*. Mathematically: `FP[IP[i]] = i` for all i = 1…64

Application

Apply FP to the 64-bit R16 // L16 value — each output bit position is selected from the input at the position given in the FP table.

Pattern of FP

FP re-interleaves the separated even/odd bits that IP originally split apart — restoring byte-aligned data for hardware output.

Result

FP output = **Final 64-bit Ciphertext**

Applying FP: Step-by-Step Output Bits

INPUT: R16 // L16 (64 BITS) → FP → CIPHERTEXT

Input to FP (R16 // L16)

```
Pos:  1  2  3  4  5  6  7  8   9 10 11 12 13 14 15 16  ...
Bit:  0  0  0  0  1  0  1  0   0  1  0  0  1  1  0  0  ...
```

First 8 Output Bits (FP positions: 40,8,48,16,56,24,64,32)

Out Pos	Reads Input Bit	Bit Value
1	40	1
2	8	0
3	48	0
4	16	0
5	56	0
6	24	1
7	64	0
8	32	1

First 8 output bits = `10000101` = hex `85` — matching the first byte of our expected ciphertext `85E813540F0AB405` ✓

Continuing FP for all 64 positions produces the complete 64-bit ciphertext shown on the next slide.

Final Permutation: Generating the Ciphertext

NIST DES TEST VECTOR — VERIFIED RESULT

Input to FP (R16 // L16, 64 bits)

```
0000 1010  0100 1100  1101 1001  1001 0101  0100 0011  0100 0010  0011 0010  0011 0100
```

After Applying All 64 FP Positions

```
Final Ciphertext (64 bits):
1000 0101  1110 1000  0001 0011  0101 0100  0000 1111  0000 1010  1011 0100  0000 0101
```

Ciphertext Bytes

85 | E8 | 13 | 54 | 0F | 0A | B4 | 05

Final Ciphertext (Hex)

85E813540F0AB405

Verification ✓

Matches published NIST DES test vector exactly — algorithm correct

Complete Verification: Plaintext 0123456789ABCDEF + Key 133457799BBCDFF1 → Ciphertext 85E813540F0AB405 ✓ (matches NIST FIPS PUB 46 test vector)

Complete Encryption Summary: End-to-End Flow

01

IP Applied to Plaintext

`0123456789ABCDEF` → permuted → L0=`CC00CCFF`, R0=`F0AAF0AA`

02

Key Schedule Generated

Key `133457799BBCDFF1` → PC-1 → C0,D0 → 16 shifts → PC-2 → K1…K16 (each 48 bits)

03

16 Feistel Rounds

Each: Li=Ri-1, Ri=Li-1⊕F(Ri-1,Ki); where F = E(32→48) → XOR → 8 S-Boxes(48→32) → P(32→32)

04

Final Swap + FP

R16 // L16 → Apply IP^{-1} → 64-bit ciphertext `85E813540F0AB405`

Operation Count (Full DES Encryption)

Operation	Count	Notes
Permutations (IP + FP)	2	Fixed tables, hardware-optimized
E-Expansions (per round)	16	32→48 bit expansion
XOR operations	32	16 inside F + 16 Feistel combinations
S-Box lookups	128	8 per round × 16 rounds
P-Permutations	16	32→32 inside each F-function
Key schedule ops	32	16 shifts + 16 PC-2 selections

DES Decryption: Same Algorithm, Reversed Keys

The Core Principle

DES decryption uses the **identical algorithm** as encryption. The *only* difference is the key schedule is applied in **reverse order**: K16, K15, K14, ..., K2, K1.

Why This Works: Feistel Proof

Encryption:
`Li = Ri-1`
`Ri = Li-1 ⊕ F(Ri-1, Ki)`

Recovery (decryption):
Given Li and Ri, recover:
`Ri-1 = Li`
`Li-1 = Ri ⊕ F(Li, Ki)`

F does **NOT** need to be invertible — XOR self-inverse handles recovery automatically.

Hardware Advantage

Same DES chip can encrypt and decrypt — just reverse the key loading order. Reduces implementation cost by 50%.

Decryption Flow

01

Input: Ciphertext

`85E813540F0AB405`

02

Apply IP

Get L'0 // R'0 = R16 // L16 from encryption

03

16 Rounds: K16→K1

Same Feistel equations; keys in reverse order

04

Final Swap + FP

Recover original plaintext `0123456789ABCDEF` ✓

Decryption: Round-by-Round Key Reversal

DECRYPTION KEY ORDER: K16 → K15 → … → K2 → K1

Decryption Round	Key Used	Recovers from Encryption
1 (Dec)	K16	R15 and L15
2 (Dec)	K15	R14 and L14
3 (Dec)	K14	R13 and L13
… (Dec)	…	… continuing backwards …
15 (Dec)	K2	R1 and L1
16 (Dec)	K1	R0 and L0 = original halves

Decryption Round 1 Detail (uses K16)

Input: L'0 = R16, R'0 = L16 (from final swap reversal)
`L'1 = R'0; R'1 = L'0 ⊕ F(R'0, K16)`
This recovers R15 and L15 from encryption

Mathematical Guarantee

For any plaintext P and key K:
`DEC_K(ENC_K(P)) = P`
Proven by the Feistel structure's self-inverse property — not merely claimed

3DES Extension

`ENC_K1(DEC_K2(ENC_K3(P)))`
Applies DES three times with different keys for **112-bit effective security**

Decryption: Verification with Complete Example

FULL ROUND-TRIP VERIFICATION — NIST TEST VECTOR

Step	Operation	Result
Input	Ciphertext	85E813540F0AB405
After IP	Apply Initial Permutation	L'0 = 0A4CD995, R'0 = 43423234 = R16 // L16 ✓
Rounds 1–16	Apply K16→K1	Progressively reverses each encryption round
After Round 16	Final decryption round	L'16 = CC00CCFF, R'16 = F0AAF0AA = L0 // R0 ✓
Final Swap	Combine R'16 // L'16	F0AAF0AA // CC00CCFF
After FP	Apply IP^{-1}	0123456789ABCDEF ✓ Original plaintext recovered!

Decryption Confirmed ✓

Original plaintext `0123456789ABCDEF` recovered exactly

Feistel Elegance

Encryption and decryption are the same operation — only the key order changes. One circuit, two functions

CBC Mode Note

In CBC mode, also XOR each decrypted block with the previous ciphertext block (or IV for block 1)

DES Mathematical Summary: All Formulas

Initial Permutation

$L0 \;//\; R0 = IP(\text{Plaintext})$ — fixed 64-bit permutation, no cryptographic strength

Feistel Round (i = 1 to 16)

$L_i = R_{i-1}$
$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$

F-Function (full decomposition)

$F(R, K) = P(\, S(\, E(R) \oplus K \,) \,)$
E(R): 32→48 | ⊕K: 48-bit XOR | S(·): 48→32 (8 S-Boxes) | P(·): 32→32

S-Box Lookup Rule

For 6-bit input b1b2b3b4b5b6:
Row = $2 \times b1 + b6$ (0–3) | Column = $8 \times b2 + 4 \times b3 + 2 \times b4 + b5$ (0–15) | Output = S[row][col] (4 bits)

Key Schedule Formula

$K_i = PC\text{-}2(\, LSi(C_{i-1}) \;//\; LSi(D_{i-1}) \,)$
LSi = left circular shift by 1 or 2 per schedule

Final Output

$\text{Ciphertext} = FP(R16 \;//\; L16) = IP^{-1}(R16 \;//\; L16)$

Decryption

Same as encryption with keys K16, K15, ..., K1
$DEC_K(ENC_K(P)) = P \;\forall\; P, K$

Advantages of DES

Speed in Hardware

All operations map directly to simple logic gates — permutations, XOR, S-Box lookups. 1970s hardware encrypted at 1 Mbps; modern silicon is orders of magnitude faster.

40+ Years of Cryptanalysis

Best known attacks: differential cryptanalysis (Biham & Shamir, 1990) and linear cryptanalysis (Matsui, 1993) — both require 2^{47} chosen plaintexts, impractical in most scenarios.

Strong Avalanche Effect

Changing 1 plaintext bit changes ~50% of ciphertext bits after just 5 rounds — full diffusion achieved well before Round 16.

Complete Specification

Fully specified in a single FIPS document — no ambiguity, no implementation choices. Any two correct DES implementations produce **identical output** for the same input and key.

Feistel Elegance

Same hardware/software for encryption and decryption — reduces implementation cost by 50%. The self-inverse property is mathematically proven, not assumed.

Foundation for AES

DES's Feistel structure and design principles directly influenced AES and most modern block ciphers. It remains the canonical teaching example in every cryptography course worldwide.

Disadvantages of DES

56-bit Key Too Short

$2^{56} \approx 72$ quadrillion keys — sounds large, but EFF's "Deep Crack" (1998) broke DES in **56 hours** for $250,000. Today, cloud computing makes it even cheaper and faster.

Officially Broken & Deprecated

By 2006, Copacobana (FPGA cluster) broke DES in 9 days for $10,000. NIST officially withdrew FIPS 46-3 in **2005**. Any system using single DES today is considered cryptographically broken.

64-bit Block Size Weakness

Small block size enables birthday attacks in modes like CBC after only 2^{32} blocks (~32 GB of data). The **SWEET32 attack (2016)** exploits this — breaking 64-bit block ciphers in TLS with enough traffic.

Weak and Semi-Weak Keys

4 weak keys (e.g., `0x0101010101010101`) where K1=K2=...=K16 — encryption = decryption. **12 semi-weak key pairs** also exist where Ei(P,K1) = Di(P,K2).

Complementation Property

$ENC(\bar{P}, \bar{K}) = \overline{ENC(P, K)}$ — halves the effective brute-force search space from 2^{56} to 2^{55}. An attacker can test complementary key pairs simultaneously.

NSA S-Box Controversy

NSA's 1975 S-Box modifications were unexplained until 1990 — raised concerns about backdoors for a decade. Biham & Shamir later confirmed the changes *strengthened* DES against differential cryptanalysis.

DES vs AES: Comprehensive Comparison

Parameter	DES	AES
Year Published	1977	2001
Block Size	64 bits	128 bits
Key Size	56 bits (effective)	**128 / 192 / 256 bits**
Number of Rounds	16	10 / 12 / 14 (key-dependent)
Structure	Feistel Network	Substitution-Permutation Network (SPN)
S-Box Size	6-bit in → 4-bit out (8 S-Boxes)	8-bit in → 8-bit out (1 S-Box, reused)
Key Schedule	PC-1, circular shifts, PC-2	Key Expansion with Rcon constants
Security Status	**Broken** (1998)	Secure — no known practical attack
Brute Force Time	Hours (modern hardware)	2^{128} operations — computationally infeasible
Hardware Performance	Very fast (simple gates)	Very fast (AES-NI CPU instructions)
Standard	FIPS 46 (withdrawn 2005)	FIPS 197 (current)
Applications	Legacy only; 3DES transitional until 2024	TLS, AES-GCM, disk encryption, VPN, SSH

Real-World Applications of DES

ATM PIN Encryption

ATM PIN encryption used DES from 1977–2000s. **PIN Block Format 0 (ISO 9564)** encrypted 4–12 digit PINs before transmission to bank host. Hardware Security Modules in ATMs still support DES for backward compatibility.

Smart Cards & EMV

Early EMV chip cards (pre-2010) used **3DES** for card authentication and session key derivation. Still present in some transit cards (London Oyster, early Mifare Classic implementations).

Satellite Communications

Military and commercial satellite communications used DES for link encryption throughout the 1980s–1990s. Some legacy ground station equipment still supports it for interoperability.

Hardware Security Modules

IBM 4758 cryptographic coprocessor, Thales HSMs — all support DES/3DES for legacy application compatibility. Enterprise HSMs must maintain backward compatibility with decades-old systems.

Triple DES (3DES)

`ENC_K1(DEC_K2(ENC_K3(P)))` — extends DES to **112-bit effective security**. Still used in payment card industry; PCI DSS allowed 3DES until 2024.

Academic Foundation

DES remains the canonical teaching example for block cipher design. Every cryptography course uses it to explain Feistel networks, S-Box construction, and key scheduling.

Class Activity: Hands-On DES Round

WORK IN GROUPS OF 3 — SHOW ALL BIT-LEVEL WORKING

Given Values

Plaintext (hex): ABCDEF1234567890
Key (hex): AABB09182736CCDD

Time Allocation

Tasks 1–3: 10 min | Tasks 4–6: 15 min | Tasks 7–8: 10 min

1

Hex to Binary

Convert plaintext and key to 64-bit binary — show all 64 bits for each

2

Initial Permutation

Apply IP table to plaintext binary; identify L0 and R0; express in hex

3

Key Schedule

Apply PC-1 to key → C0, D0; perform Round 1 shift (1 position); apply PC-2 → K1

4

Expansion

Apply E-table to R0; show all 48 bits of E(R0) grouped as 8 × 6 bits

5

XOR

Compute E(R0) ⊕ K1 bit-by-bit; show all 8 groups of 6 bits with working

6

S-Box Lookup

For each of the 8 groups: identify row (b1b6) and column (b2b3b4b5); look up S-Box value; convert to 4-bit binary

7

P-Permutation

Apply P-table to 32-bit S-Box output to get F(R0, K1)

8

Round 1 Output

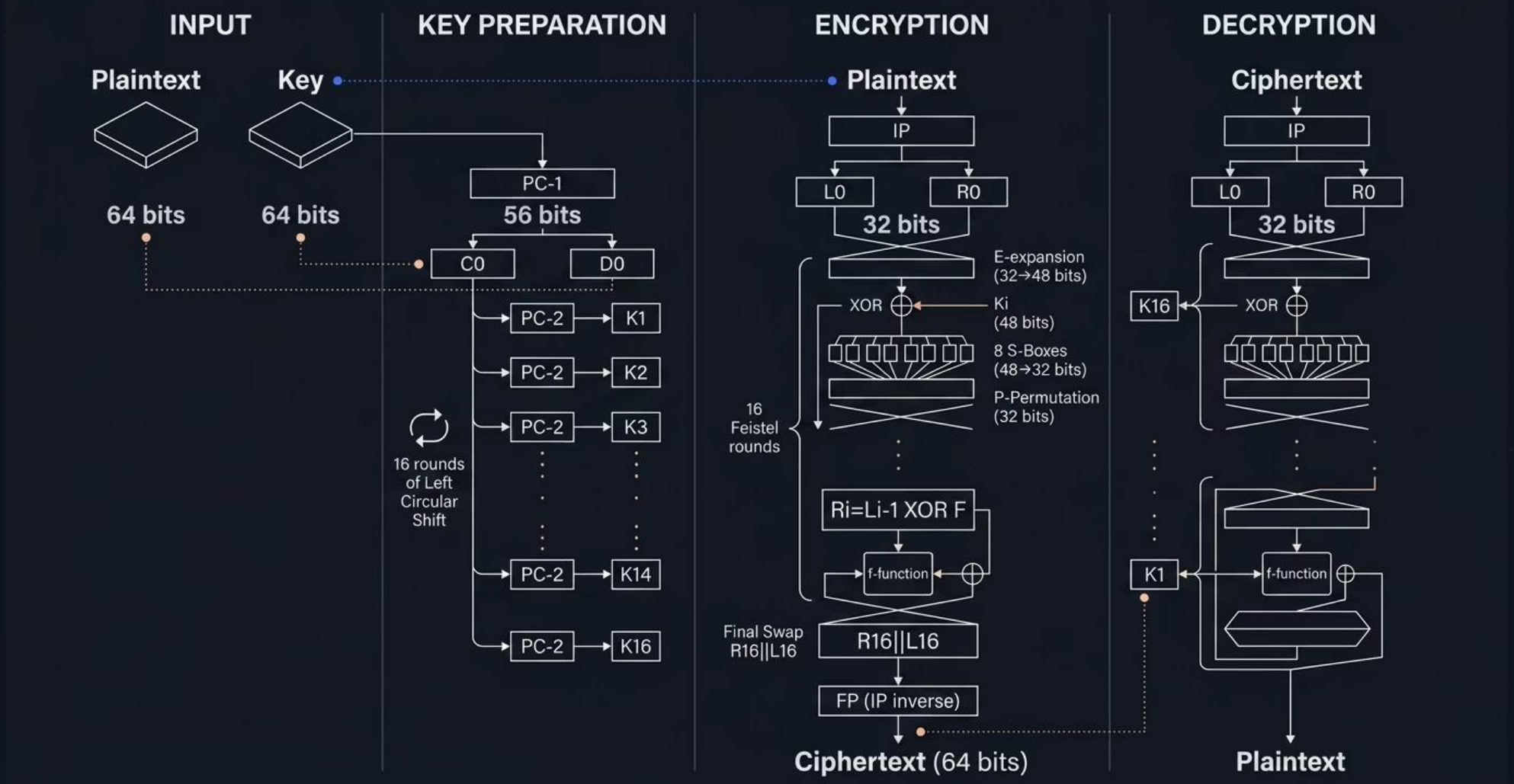
Compute R1 = L0 ⊕ F(R0,K1) and L1 = R0; express both in hex

Quiz: Test Your DES Knowledge

10 QUESTIONS — COVER YOUR ANSWERS AND TEST YOURSELF FIRST!

#	Question	Answer
Q1	What is the effective key length of DES?	**56 bits** (8 parity bits discarded)
Q2	How many rounds does DES perform?	**16 Feistel rounds**
Q3	What does the Expansion function do?	**Expands 32 bits → 48 bits** by duplicating boundary bits
Q4	In S-Box lookup, how is the row determined?	**Bits b1 and b6** (first and last of the 6-bit input): row = 2×b1 + b6
Q5	What is the output size of each S-Box?	**4 bits** — 6-bit input → 4-bit output; 8 S-Boxes → 32 bits total
Q6	What is FP in DES?	**IP^{-1}** — the inverse of the Initial Permutation; undoes IP
Q7	How does DES decryption differ from encryption?	**Same algorithm; keys applied in reverse order** K16→K1
Q8	What property makes DES decryption possible without inverting F?	**XOR self-inverse property**: $(A \oplus B) \oplus B = A$ — no F^{-1} needed
Q9	What year was DES officially broken by brute force?	**1998** — EFF Deep Crack, 56 hours, $250,000
Q10	What replaced DES as the U.S. encryption standard?	**AES (Advanced Encryption Standard)**, adopted 2001, FIPS 197

Complete DES Process: Animated Summary Flowchart



Data Path Summary

Plaintext → **IP** → L0/R0 → **16×**(E→XOR→S-Box→P→Feistel) → Swap → **FP** → Ciphertext

Key Schedule Summary

Key → **PC-1** → C0/D0 → **16×**(LS shift + PC-2) → K1…K16 (each 48 bits)

Decryption

Same flow, same tables — only key order reversed: **K16 → K1**

Thank You — Questions?

Core Architecture

DES = 64-bit block, 56-bit key, 16 Feistel rounds — the landmark in cryptographic standardization that shaped the entire field

Security Principles

Security = **Confusion** (S-Boxes) + **Diffusion** (permutations) + **Key mixing** (XOR) — Shannon's properties embodied in hardware

Feistel Elegance

Same algorithm encrypts and decrypts — just reverse the key order. F need not be invertible; XOR self-inverse handles everything

Legacy & Relevance

DES is broken for modern use — but its design principles live in AES, 3DES, and every modern block cipher in production today

---

Next Topics in This Course

1

Triple DES (3DES)

ENC-DEC-ENC with multiple keys → 112-bit security

2

AES (Rijndael)

SPN structure, MixColumns, ShiftRows, 128-bit blocks

3

Modes of Operation

ECB, CBC, CTR, GCM — how block ciphers handle long messages

Further Reading

→ **FIPS PUB 46-3**

Data Encryption Standard — original NIST specification with all tables and test vectors

→ **Stallings, W.**

Cryptography and Network Security, Chapter 3 — comprehensive DES walkthrough

→ **Schneier, B.**

Applied Cryptography, Chapter 12 — practical perspective on DES security

→ **NIST SP 800-67**

Recommendation for Triple DES Block Cipher — current 3DES guidance